

Model Catalog

1. Purpose

This document lists the models and model-like components currently used in the repository and explains:

- what each one does
- why it exists
- where it is used
- what its strengths are
- what it cannot do well yet
- what inputs or supporting details it needs

The repository uses a mix of neural networks, lightweight statistical models, and policy models. Not all of them are prediction models in the strict sense, but all of them influence the system's behavior.

2.1 Current validation snapshot

The latest comparison snapshot is useful as a quick ordering reference, but it should be read alongside the full validation and holdout matrices in the dashboard and docs.

The standalone web dashboard uses display-only label swapping in some model-statistics panels, so vit and transformer can appear exchanged in the UI even though the underlying model identifiers remain distinct.

The selection value used by the dashboard ranking is a weighted score:

$\text{selection_value} = (0.5 \text{ CV F1}) + (0.3 \text{ CV Accuracy}) + (0.2 * \text{ CV Precision}) + \text{model priority bonus}$

Rank	Model	CV Accuracy	CV Precision	CV Recall	CV F1	CV Balanced Acc
1	`multimodal_attention`	95.0%	93.3%	100.0%	96.0%	95.0%
2	`transformer`	91.7%	87.8%	93.3%	89.8%	91.7%
3	`vit`	80.0%	76.7%	86.7%	80.2%	80.0%

The current docs treat this as the active three-model comparison set. The older CNN and LSTM baselines are kept in the catalog below as legacy references.

2. Quick Summary Table

Model / Component	Type	Purpose	Advantage
`HandwritingEncoder`	CNN encoder	Convert handwriting images into feature vectors	Fast, simple,
`AudioEncoder`	1D CNN encoder	Convert spectral audio features into embeddings	Compact and
`TextEncoder`	BiGRU encoder	Encode character sequences	Good baseline
`LSTMTextEncoder`	BiLSTM encoder	Encode text with recurrent memory	Useful baseline

Model / Component	Type	Purpose	Advantage
`TransformerTextEncoder`	Transformer encoder	Encode text with attention and positional context	Stronger sequence modeling
`ViTHandwritingEncoder`	Vision transformer	Learn handwriting from image patches	Patch-level attention
`BehaviorEncoder`	MLP encoder	Encode reading-behavior numeric features	Simple and fast
`FusionClassifier`	Multimodal classifier	Fuse handwriting, audio, text, behavior, and errors	Flexible and modular
`AttentionFusionClassifier`	Multimodal attention classifier	Learn modality importance weights	Interpretable fusion
`InitialCNNModel`	Legacy baseline classifier	Compact image/audio baseline	Lightweight
`InitialLSTMModel`	Legacy baseline classifier	Text-focused baseline	Useful when text is primary
`InitialCNNLSTMModel`	Legacy baseline multimodal	Early multimodal baseline	Combines image and text
`MultimodalDyslexiaModel`	Default screening model	Main multimodal risk classifier	Balanced and robust
`TransformerMultimodalModel`	Multimodal classifier	Transformer-based text branch	Better context modeling
`ViTMultimodalModel`	Multimodal classifier	Vision-transformer handwriting branch	Stronger image features
`ViTTransformerMultimodalModel`	Multimodal classifier	Transformer-heavy multimodal model	Strongest sequence modeling
`AttentionMultimodalModel`	Multimodal classifier	Attention-weighted modality fusion	Exposes modality strengths
`BengaliLearningDisorderFoundationModel`	Foundation model	Shared representation learning across tasks	Reusable across tasks
`LearningDisorderAdapter`	Adapter head	Task-specific head on foundation embeddings	Efficient fine-tuning
`AudioContrastiveModel`	SSL model	Learn audio invariances	Good for pre-training
`AudioMaskedReconstructionModel`	SSL model	Reconstruct masked audio regions	Learns local audio structure
`AudioTeacherDistillModel`	SSL model	Distill teacher audio knowledge	Can reuse a pre-trained model
`LogisticRegression` in biomarker discovery	Statistical model	Rank biomarker importance	Interpretable results
`InterventionPolicy`	Q-table policy	Choose intervention action	Simple, editable
`AdaptiveTutorAgent`	Q-table policy	Adaptive tutoring action selection	Lightweight

3. Neural Encoder Components

3.1 HandwritingEncoder

Found in:

- `src/dyslexiadetection/models.py`
(`/d:/Project/DyslexiaDetectionSystem/src/dyslexiadetection/models.py`)

Functionality:

- takes grayscale handwriting images
- applies stacked convolutions and pooling
- compresses the image into a fixed-size embedding

Purpose:

- learn stroke and shape patterns from handwriting
- support dysgraphia/dyslexia-related visual cues

Advantages:

- fast
- easy to train
- works well with local CPU inference

Gaps:

- only sees normalized grayscale inputs
- does not explicitly model pen pressure or stroke order

Required details:

- fixed-size handwriting image
- preprocessing that normalizes size and contrast

3.2 AudioEncoder

Found in:

- `src/dyslexiadetection/models.py`
(`/d:/Project/DyslexiaDetectionSystem/src/dyslexiadetection/models.py`)

Functionality:

- consumes audio spectral features
- uses 1D convolutions to learn temporal patterns

Purpose:

- capture fluency, pause structure, and energy-related cues

Advantages:

- compact
- suitable for local use
- works directly on extracted spectral features

Gaps:

- depends heavily on audio preprocessing quality
- does not directly model raw waveform timing as richly as larger audio networks

Required details:

- mono audio converted into spectral features
- stable sample rate and frame length

3.3 TextEncoder

Found in:

- `src/dyslexiadetection/models.py`
(`/d:/Project/DyslexiaDetectionSystem/src/dyslexiadetection/models.py`)

Functionality:

- encodes character sequences with a bidirectional GRU

Purpose:

- model short reading or spelling text with modest complexity

Advantages:

- efficient
- strong baseline for character-level text
- works well for multilingual token sequences

Gaps:

- may lose long-range dependencies
- not as expressive as transformer encoders

Required details:

- character-tokenized sequence
- consistent vocabulary from the chosen language

3.4 LSTMTextEncoder

Found in:

- `src/dyslexiadetection/models.py`
(`/d:/Project/DyslexiaDetectionSystem/src/dyslexiadetection/models.py`)

Functionality:

- encodes character sequences with a bidirectional LSTM

Purpose:

- text baseline for sequence modeling

Advantages:

- familiar recurrent behavior
- robust for small text samples

Gaps:

- limited compared with attention-based encoders

Required details:

- tokenized sequence
- language-specific vocabulary

3.5 TransformerTextEncoder

Found in:

- `src/dyslexiadetection/models.py`
(`/d:/Project/DyslexiaDetectionSystem/src/dyslexiadetection/models.py`)

Functionality:

- uses embeddings, learned positional vectors, and transformer layers
- masks padding tokens during pooling

Purpose:

- stronger text representation where token context matters

Advantages:

- handles order and context better
- supports attention-based interpretation

Gaps:

- slightly heavier than recurrent text encoders
- still character-level rather than semantic word-level

Required details:

- fixed maximum text length
- padded token sequence

3.6 ViTHandwritingEncoder

Found in:

- `src/dyslexiadetection/models.py`
(/d:/Project/DyslexiaDetectionSystem/src/dyslexiadetection/models.py)

Functionality:

- splits handwriting images into patches
- uses transformer encoding over patch tokens

Purpose:

- learn handwriting layout and patch-level relationships

Advantages:

- better access to global image structure
- supports attention visualizations

Gaps:

- more compute-intensive than CNN handwriting encoding
- image size must be divisible by patch size

Required details:

- fixed image size
- patch-compatible dimensions

3.7 BehaviorEncoder

Found in:

- `src/dyslexiadetection/models.py`
(/d:/Project/DyslexiaDetectionSystem/src/dyslexiadetection/models.py)

Functionality:

- transforms reading behavior counts into a learned feature vector

Purpose:

- incorporate timing and fluency indicators into the model

Advantages:

- simple
- strong fit for low-dimensional numeric features

Gaps:

- depends on clean numeric logging
- cannot recover missing behavior signals by itself

Required details:

- reading time
- hesitations
- repetitions
- omissions

4. Multimodal Screening Models

The current active supervised ranking set is the three-model trio above. The remaining models here are still useful for understanding the system and for historical comparison, but they are not the main ranked set in the current docs.

4.1 FusionClassifier

Found in:

- `src/dyslexiadetection/models.py`
(`/d:/Project/DyslexiaDetectionSystem/src/dyslexiadetection/models.py`)

Functionality:

- encodes each modality separately
- concatenates the embeddings and error vector
- classifies with a dense head

Purpose:

- provide the standard multimodal fusion mechanism

Advantages:

- easy to inspect
- easy to extend
- reusable across multiple architectures

Gaps:

- treats modalities mostly as a concatenated block
- does not dynamically reweight modalities

Required details:

- image tensor
- audio tensor
- text tensor
- error counts
- behavior vector when available

4.2 AttentionFusionClassifier

Found in:

- `src/dyslexiadetection/models.py`
(`/d:/Project/DyslexiaDetectionSystem/src/dyslexiadetection/models.py`)

Functionality:

- projects each modality into a common space
- scores each modality with a learned attention head
- computes a weighted fusion

Purpose:

- expose modality importance in addition to prediction

Advantages:

- more interpretable than raw concatenation
- can reveal which modality dominated a prediction

Gaps:

- more parameters
- attention weights are useful but still not a full explanation by themselves

Required details:

- all input modalities
- optional behavior vector

4.3 InitialCNNModel

Found in:

- `src/dyslexiadetection/models.py`
(`/d:/Project/DyslexiaDetectionSystem/src/dyslexiadetection/models.py`)

Functionality:

- combines handwriting and audio with error counts

Purpose:

- compact baseline for early experiments

Advantages:

- smallest multimodal-style model in the catalog
- fast inference

Gaps:

- ignores text sequence modeling
- ignores behavior vector

Required details:

- image
- audio
- spelling/pronunciation errors

4.4 InitialLSTMModel

Found in:

- `src/dyslexiadetection/models.py`
(`/d:/Project/DyslexiaDetectionSystem/src/dyslexiadetection/models.py`)

Functionality:

- combines text, behavior, and error counts

Purpose:

- text-centric baseline

Advantages:

- simple
- useful when audio/image inputs are unavailable

Gaps:

- ignores handwriting and audio

Required details:

- text sequence
- behavior vector
- error counts

4.5 InitialCNNLSTMMModel

Found in:

- `src/dyslexiadetection/models.py`
(`/d:/Project/DyslexiaDetectionSystem/src/dyslexiadetection/models.py`)

Functionality:

- joins handwriting, audio, text, behavior, and error features

Purpose:

- early full multimodal baseline

Advantages:

- broader evidence coverage
- still lightweight enough for experimentation

Gaps:

- shallower than the newer transformer/attention models

Required details:

- every modality tensor used by the main screening stack

4.6 MultimodalDyslexiaModel

Found in:

- `src/dyslexiadetection/models.py`
(`/d:/Project/DyslexiaDetectionSystem/src/dyslexiadetection/models.py`)

Functionality:

- the default multimodal screening model
- uses CNN handwriting encoding, audio encoding, GRU text encoding, and behavior encoding

Purpose:

- main risk classifier for the standard screening workflow

Advantages:

- balanced feature coverage
- practical default choice
- easy to train on modest datasets

Gaps:

- concatenation fusion is less adaptive than attention fusion

Required details:

- handwriting image
- reading audio
- text sample
- spelling/pronunciation error counts
- behavior counts

4.7 TransformerMultimodalModel

Found in:

- `src/dyslexiadetection/models.py`
(`/d:/Project/DyslexiaDetectionSystem/src/dyslexiadetection/models.py`)

Functionality:

- replaces the GRU text branch with a transformer encoder

Purpose:

- improve text-sequence modeling

Advantages:

- better contextual awareness
- often stronger on structured text inputs

Gaps:

- heavier than the baseline multimodal model

Required details:

- same as the default multimodal model

4.8 ViTMultimodalModel

Found in:

- `src/dyslexiadetection/models.py`
(`/d:/Project/DyslexiaDetectionSystem/src/dyslexiadetection/models.py`)

Functionality:

- replaces the CNN handwriting branch with a vision transformer encoder

Purpose:

- improve handwriting structure modeling

Advantages:

- patch-level representation
- better fit for attention-based image explanation

Gaps:

- more demanding in compute and tuning

Required details:

- fixed-size handwriting image
- all other standard multimodal inputs

4.9 ViTTransformerMultimodalModel

Found in:

- src/dyslexiadetection/models.py
(/d:/Project/DyslexiaDetectionSystem/src/dyslexiadetection/models.py)

Functionality:

- uses ViT for handwriting and transformer for text

Purpose:

- strongest standard multimodal transformer combination in the project

Advantages:

- strongest modeling flexibility among the standard supervised variants

Gaps:

- highest complexity among the regular model families

Required details:

- all modalities
- consistent preprocessing and vocabulary

4.10 AttentionMultimodalModel

Found in:

- src/dyslexiadetection/models.py
(/d:/Project/DyslexiaDetectionSystem/src/dyslexiadetection/models.py)

Functionality:

- applies learned modality attention before classification

Purpose:

- expose modality importance and improve interpretability

Advantages:

- more informative than plain concatenation

Gaps:

- attention weights should be interpreted carefully

Required details:

- all modalities used in the main multimodal pipeline

5. Foundation and Transfer Models

5.1 BengaliLearningDisorderFoundationModel

Found in:

- `src/dyslexiadetection/foundation.py`
(`/d:/Project/DyslexiaDetectionSystem/src/dyslexiadetection/foundation.py`)

Functionality:

- encodes handwriting, audio, text, and behavior into a shared latent space
- applies multimodal projection and normalization
- supports contrastive, reconstruction, and masked-text objectives

Purpose:

- learn a reusable base representation for related learning-disorder tasks

Advantages:

- reusable across tasks
- better fit for low-resource adaptation
- can support transfer to dyslexia, dysgraphia, and dyscalculia heads

Gaps:

- needs substantial pretraining data to realize its full benefit
- more complex to train than the standard screening model

Required details:

- all four modalities
- error features
- a pretraining objective

5.2 LearningDisorderAdapter

Found in:

- `src/dyslexiadetection/foundation.py`
(`/d:/Project/DyslexiaDetectionSystem/src/dyslexiadetection/foundation.py`)

Functionality:

- attaches a disorder-specific prediction head to the foundation model

Purpose:

- fine-tune the shared backbone for a specific disorder

Advantages:

- efficient adaptation
- avoids retraining the full foundation model

Gaps:

- depends on the quality of the foundation embedding

Required details:

- foundation checkpoint
- target disorder name
- task labels

5.3 Cross-lingual transfer helper

Found in:

- `src/dyslexiadetection/crosslingual.py`
(`/d:/Project/DyslexiaDetectionSystem/src/dyslexiadetection/crosslingual.py`)

Functionality:

- copies matching parameters from a source checkpoint
- can freeze selected branches
- supports feature distillation

Purpose:

- reuse source-language knowledge in a target language

Advantages:

- very helpful for low-resource Bengali fine-tuning

Gaps:

- only transfers tensors that match in name and shape

Required details:

- source checkpoint
- target model with compatible modules
- prefix list for transfer

6. Self-Supervised Audio Models

These appear in `src/dyslexiadetection/sslpretraining.py`
(`/d:/Project/DyslexiaDetectionSystem/src/dyslexiadetection/sslpretraining.py`).

6.1 AudioContrastiveModel

Functionality:

- learns embeddings from two augmented audio views
- uses a contrastive loss

Purpose:

- pretrain useful audio representations without labels

Advantages:

- effective when labels are limited

Gaps:

- requires good augmentation design

Required details:

- audio feature tensors
- augmentation pipeline

6.2 AudioMaskedReconstructionModel

Functionality:

- reconstructs masked spectral regions

Purpose:

- force the encoder to learn local temporal/audio structure

Advantages:

- simple SSL objective

Gaps:

- performance depends on masking strategy

Required details:

- audio features
- masking parameters

6.3 AudioTeacherDistillModel

Functionality:

- learns from a teacher embedding or teacher-guided signal

Purpose:

- transfer stronger audio knowledge to a smaller student model

Advantages:

- good when a teacher model already exists

Gaps:

- depends on a meaningful teacher checkpoint

Required details:

- student audio input
- teacher checkpoint or teacher embedding source

7. Statistical and Policy Models

7.1 LogisticRegression in biomarker discovery

Found in:

- `src/dyslexiadetection/biomarkers.py`
(`/d:/Project/DyslexiaDetectionSystem/src/dyslexiadetection/biomarkers.py`)

Functionality:

- fits a simple linear classifier over biomarker features
- ranks coefficients as one signal of importance

Purpose:

- help rank candidate biomarkers with an interpretable statistical baseline

Advantages:

- fast
- interpretable
- low overhead

Gaps:

- only captures linear relationships

Required details:

- numeric biomarker matrix
- binary or near-binary label structure

7.2 InterventionPolicy

Found in:

- `src/dyslexiadetection/intervention.py`
(`/d:/Project/DyslexiaDetectionSystem/src/dyslexiadetection/intervention.py`)

Functionality:

- maintains a Q-table over learner states and intervention actions

Purpose:

- choose the next intervention plan based on the learner profile

Advantages:

- transparent
- easy to save and load
- easy to debug

Gaps:

- handcrafted state space
- not a neural policy

Required details:

- severity level
- error counts
- reading duration
- fluency disruption counts

7.3 AdaptiveTutorAgent

Found in:

- `src/dyslexiadetection/adaptivetutoring.py`
(`/d:/Project/DyslexiaDetectionSystem/src/dyslexiadetection/adaptivetutoring.py`)

Functionality:

- selects tutoring actions using an epsilon-greedy Q-learning style policy

Purpose:

- adapt practice actions over time based on observed reward

Advantages:

- lightweight
- persistent
- easy to inspect

Gaps:

- depends on a handcrafted state definition
- reward shaping strongly affects behavior

Required details:

- state features

- action list
- reward signal

8. Which Model To Use When

Screening

Best default:

- MultimodalDyslexiaModel

Good alternatives:

- TransformerMultimodalModel
- AttentionMultimodalModel
- ViTTransformerMultimodalModel

Lightweight baseline experiments

Use:

- InitialCNNModel
- InitialLSTMModel
- InitialCNNLSTMModel

Cross-lingual or low-resource adaptation

Use:

- BengaliLearningDisorderFoundationModel
- LearningDisorderAdapter
- cross-lingual weight transfer helpers

Audio-only representation learning

Use:

- AudioContrastiveModel
- AudioMaskedReconstructionModel
- AudioTeacherDistillModel

Biomarker analysis

Use:

- LogisticRegression plus the biomarker feature builder

Intervention and tutoring

Use:

- InterventionPolicy
- AdaptiveTutorAgent

9. Common Gaps Across The Catalog

Across most model families, the main limitations are:

- they depend on clean input preparation
- they assume the manifest and behavior values are present and meaningful
- they are built for screening and assistance, not diagnosis
- some models are intentionally compact, so they trade expressive power for local efficiency

10. Practical Notes For Developers

1. If you change the preprocessing, check every model that consumes that tensor shape.

1. If you change the manifest schema, check dataset loading, training, therapy, biomarker, and report code.

1. If you add a new model family, update:

- src/dyslexia_detection/models.py
- this catalog
- the architecture doc if the flow changes

1. If you remove a model, update the training CLI choices and dashboard references too.